EECS 388: Embedded Systems Lab

Final Lab Project

Objective:

In this project, you will have the opportunity to apply the knowledge and skills you have acquired in the previous labs. You will also have the opportunity to make some design choices in how you implement the final lab project.

This project centers around designing a simple embedded system that encompasses automatic braking and automatic steering functionality for a self-driving car using all the sensors that we've worked on up until this point. You will utilize the Arduino boards along with embedded components (serial communication, sensors, servos) that you have encountered so far to create a prototype that is capable of distinguishing between different breaking scenarios (obstacles which are far away, near, or very near) and is able to simulate steering using pre-determined angle values.

Project Summary:

1. **Team Composition:** This final project is designed for groups of **2 students**. Each team member must be from the **same** lab section.
2. **Total Points:** This project will be evaluated out of **50 points**.
3. **Project evaluation:** Your project will be evaluated based on the following parts:
   a. **Part 1 (35 points):** Part 1 comprises five tasks. Each task carries 7 points.
      i. **Task 1:** TFmini LiDAR
      This task involves using the TFmini Lidar sensor to collect distance measurements for objects within your surroundings. Drawing from your knowledge gained in Labs 04, you will modify the starter code to collect distance values.
      ii. **Task 2:** LED Braking
      For this task, you will use the distances values to update LEDs to indicate a certain distance. You will use knowledge gained from lab02 for this task.
      iii. **Task 3:** Board to Board Communication
      For this task, will take the distance values obtained from the TFmini LiDAR from Task1 and **transmit the distance values** to the other Arduino board.
      This task will use knowledge you gained from lab08.
      iv. **Task 4:** Steering with Servo Motor
      In this task, you must use the received angle values to drive a servo motor. Update the file from Task 1 and write the code for driving the servo using the angle values received from Raspberry Pi. Use your understanding of Lab 05 to implement this part. It may also be beneficial to rework your lab5 solution using timing interrupts that you learned in Lab9 for this.
      v. **Task 5:** Open-ended design addition
      Add an additional component to your overall project of your choosing.

Please discuss this with your GTA to make sure that the addition is reasonable in scope and accomplishable for the available time. Some possibilities are listed below in the Task Details section.

b. **Part 2 (15 points):** This part requires you to prepare a short presentation, up to 10 minutes maximum, where you talk about your project. You may use Microsoft PowerPoint, presentation board, or another tool. You do not have to demo during your presentation if you've already completed your demo with your GTA. Your presentation should cover:
    i. Your understanding of the tasks.
    ii. An explanation of how your code works
    iii. Any issues encountered while working on the project and how you overcame them.
    iv. Any improvements to the code you would make in the future and how to start on those changes.

Project Deliverables:

1. **Code:** all members of the team should submit the same **code** files to Canvas. Without the code submission, we will be unable to grade you. Submit the files directly, do not zip them.
2. **Demo:** A demonstration of all the components working together and producing the expected output. A video is sufficient for this but be sure that you have both team members in the video!
3. **Presentation:** A short presentation, up to 10 minutes maximum.

Deadlines:

The deadlines for the two parts are given below:

- **Part 1:** The demo for Part 1 is due 3 weeks from the start of final project (the start for each team is their respective lab sessions). The demo can be given any time within this period if the team has completed their project. You can also show your recorded demo videos during your presentation in week 4 if you haven't yet shown the GTA.
- **Part 2:** The presentation for Part 2 is due during lab in week 4. The presentation cannot be given earlier than this. The day of the presentation is also the last day for the demo.

We recommend that teams get their demos done ahead of time so that there is no rush on presentation day. **You can record a video or multiple short videos for your demonstration if it shows all team members**. There will be no extra time given to setup and demo during the presentation. Here is a **rough** timeline that is recommended to stay on track:

- Week 1 (Nov 9-13) – Complete task 1 & 2 during your lab session. Review tasks 3 & 4.
- Week 2 (Nov 16-20) – Complete task 3 & 4 during your lab session. Discuss a design option for task 5.
- Week 3 (Nov 23-27) – Thanksgiving – No labs this week – On your own, you can access the lab and complete task 5. You can record videos to use as demos for your GTA.
- Week 4 (Nov 30 - Dec 4) – Final Demo (if necessary) and Code is due for tasks 1-5. Present presentation during your lab session. This will be the final week of lab for EECS 388.

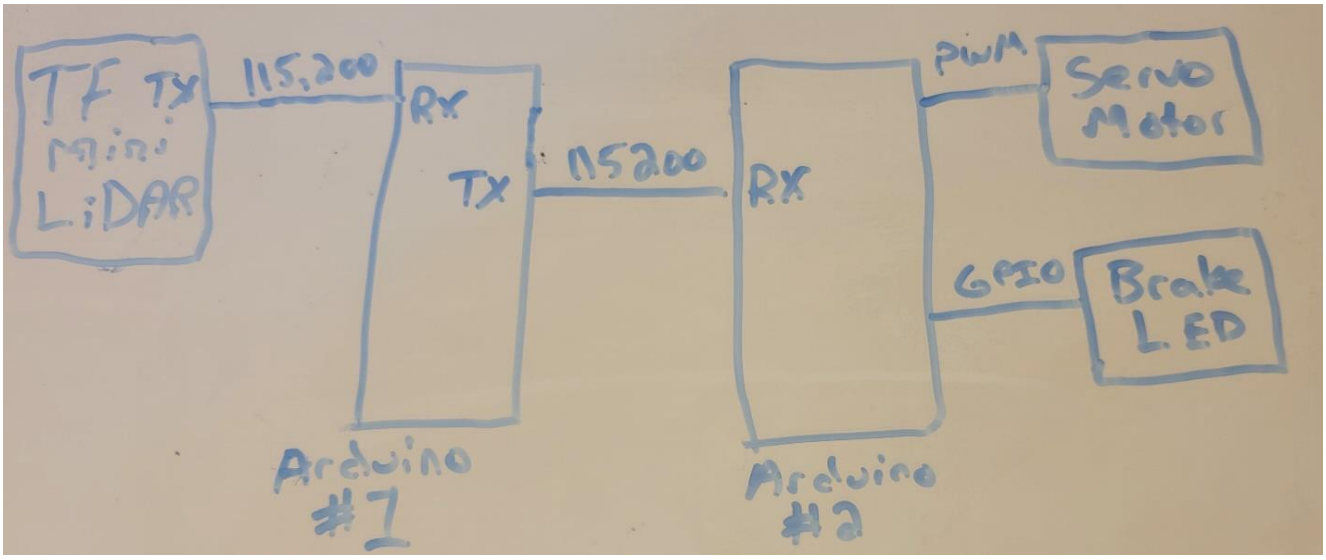
Necessary Equipment

1. Arduino Board x 2
2. TFmini Lidar Sensor
3. Servo Motor
4. Breadboard
5. LEDs, resistors

Connection Diagram

1. TFmini Lidar
    a. Green TX to Arduino1 RX
    b. Red 5V to Arduino1 5V
    c. Black GND to Arduino1 GND
2. ServoMotor
    a. Yellow PWM-> Arduino2 GPIO_6
    b. Red Power -> Arduino2 5V
    c. Brown GND -> Arduino2 GND
3. Breadboard
    a. Utilize LED circuit from Lab02
4. Arduino_1 TX to Arduino_2 RX
5. LEDs, resistors

Block Diagram



Task Details:

The **EECS_388_final_project.zip** contains the starter source code meant to run on the two Arduinos.

Task 1: Distance with LiDAR – auto_brake()

Task 1 asks you to implement a prototype automatic braking system using the TFmini lidar sensor from Lab 04. The idea behind this task is to measure the distance between our sensor (and therefore our hypothetical car) and any obstacles ahead in the direction of the sensor and decide if brakes need to be applied and how hard. You will need to configure the UART for 115200 baud speed to read distance values from the LiDAR. This will need to be done in the arduino1_lidar starter code.

Task 2: LED Breaking Indicator – auto_brake()

For the purposes of this project, we define any object that is further than 200 cm as a safe distance away and no braking is needed. If any object is less than 200 cm but more than 100 cm away, we assume that it is getting close enough to warrant braking, but only **lightly**. For anything less than 100 cm away, we assume that the object is getting too close to the vehicle, and the brakes need to be applied in full force. Finally, for something that is less than 60 cm away, we assume it is too close to us to keep the vehicle in motion and so it needs to stop and be stationary. To simulate braking, you will change the color of the LED to indicate the level of braking. The levels of braking, and their representation using LED colors, are detailed below:

Distance	Action	First LED Indicator
> 200 cm	Safe distance, no braking	Turn ON Green LED
100 cm < d ≤ 200 cm	Close, brake lightly	Turn ON Yellow LED (Red+Green)
60 < d ≤ 100 cm	Very close, brake hard	Turn ON Red LED
d ≤ 60	Too close, must stop	Turn ON *flashing* Red LED (Flashing period = 100ms)

Note, you can do this with delay() functions from inside your main loop(), but it may be advantageous to use timers and an interrupt service routine (ISR) from lab09. One way you can do this, is have your main loop() check the distance value. Inside of your timer ISR, check the distance and update the current state (SAFE_STATE, BRAKE_LIGHTLY, BRAKE_HARD, UNSAFE_STATE) and adjust the LED accordingly.

This task will need done on the arduino2_steering_brake starter code. You will need to use test values for distance to check this until you complete Task 3 (receiving the distance from arduino1_lidar).

Task 3: Board to Board Communication

Arduinos – Send distance from Arduino1 to Arduino2

You will need to write code on the Arduino1 to transmit the distance value to Arduino2. You have several design decisions to make here:

- How frequently to send the data. You can choose to send every value received from the LiDAR, or you can setup a timed task to only send it at a periodic rate (for example, 5Hz/5 times per second/every 200ms).
- How to format and send the data.
    - You can send it the same way the LiDAR does (9 bytes per frame)
    - You can choose to send only the distance_low, distance_high byte
    - You can choose to send just a single ASCII character that represents the distance, example: 'u' (Ascii value of 117 decimal to represent 117cm)

You will also need to write code on the Arduino2 to receive the distance value. You can do this with polling techniques, but remember that the Arduino2 has other things that it will need to accomplish in its main loop. This may require some experimentation to figure out how to keep things running smoothly without too much latency.

Task 4: Steering with Servomotor

This task will involve getting the servomotor setup to turn to a specific angle value. To test the function and confirm the servomotor is working, try to just turn it between 0 and 180 degrees. You can leverage your code from Lab05 for this, but remember that the main loop has other things it needs to accomplish, so using delay() functions may affect other timings.

You will need to setup an array of angle values in your Arduino code. You will loop through these values and use them as angles to drive the servomotor. Setup the following angles to loop through. Angle_values = [10, 25, 75, 45, 100, 40, 125, 15, 150, 50, 170]. These values will not be setup for you in the startup code, you will have to implement this so that it rotates through all the steering angles in this list and repeats back at the beginning.

With all this done, you should have a simple prototype for automatic braking and steering!

For the demo, you will be asked to show that your LEDs show the right color based on distance from some object, and that your servo motor responds accordingly to the specified angle values.

Task 5: Design your own addition

Work with your GTA to add an additional design element. Some options include (but are not limited to):

1. Adding a button tied as a GPIO input to the system to change the gears from Park <-> Drive mode. In Park, the steering servo is disabled and the breaking LED will indicate safe (Green), ignoring the distance from the LiDAR sensor

2. Failsafe object detection. Implement an infrared (IR) sensor as a GPIO input to the system to detect if an object is in front of the sensor. This can be a backup to the LiDAR based sensor. If the IR sensor detects something, then the LED will indicate unsafe condition and blink red.
3. Engine Temperature Warning system - Implement an engine temperate sensor using the BMP180 temperature sensor from Lab06. You will configure the I2C connection to read the temperature sensor value, and configure a secondary LED circuit to indicate the state of the engine based on the table below:

Temperature	Action	Second LED Indicator
< 70 degrees Fahrenheit	Cold Engine – Drive gently	Turn ON Blue LED
70 degrees < temp ≤ 80	Normal condition	Turn ON Green LED
Temp > 80	Engine Overheat	Turn ON Red LED

Bonus Tasks

There are several different bonus tasks available for lab10. You can choose to mix and match these however you want, but it is **highly recommended** that you get the basic pieces of your final project working first. Most of these bonus tasks won't have much starter code, you will have to go research and implement it yourself!

Bonus 1 (**2 points**) – Implement the flashing RED LED using a timer interrupt from Lab 9. Instead of changing the LEDs during the auto_brake() function, you will just set their state value (Green, Yellow, Red, or flashing Red). When the 100ms timer goes off, you will update the LEDs based on the LED state.

Bonus 2 (**2 points**) – The technique we used to control the servomotor in lab05 was manually toggling the GPIO pin to generate a PWM signal. Implement a timer-based PWM generated signal that avoids using delay() function APIs.

Bonus 3 (**5 points**) – Implement a RS232 serial interrupt service routing to handle the LiDAR communication. This will require you to research external interrupt handling for the Arduino. You can find information in the ATMega328 datasheet or online to implement this.

Bonus 4 (**5 points**) – Implement the RS232 UART communication between the two microcontrollers by using an interrupt service routine. This will require you to research external interrupt handling for the Arduino. You can find information in the ATMega328 datasheet or online to implement this.

Bonus 5 (For electrical engineers!) (**3 points**) – Implement a dedicated 5V regulated power rail for our breadboards to support the peripherals. Determine how much current our microcontroller can supply and how much is used by the TFmini LiDAR and servomotor. Demonstrate the peripherals are powered from the 5V rail and not from the microcontroller board.

Submission – 50 points

Attendance: Attendance is required for Week 1, Week 2, and Week 4 (presentation week). If you happen to complete all of the tasks during week1 and demo to your GTA, then attendance during Week 2 is optional.

Demo and Code: To obtain 35 points for this lab, you need to complete the code for tasks 1-5. Show the complete project to your GTA when you have each piece working individually and when you have it working altogether. **Note:** if you can't get all 5 tasks working together at the same time, you can take individual videos of the tasks working individually for partial credit.

Regarding video demos:

- You can use a video of each piece working individually for partial credit, but to receive full credit everything must be working together at the same time.
- Be sure your GTA knows it's you! Be sure you and your partner are visible in the video. If a GTA can't confirm it's you, then you won't get credit for the demo video.

Submit your modified eecs_388_final1.cpp and eecs_388_final2.cpp code on the "Assignments" section on Canvas under the Lab 10 submission link. Make sure you have demonstrated your lab to your GTA before you submit your code. Screenshots/PDF/Text files will not be accepted. You need to submit the modified files only.

Presentation: To obtain 15 points for this lab, you will need to prepare a short presentation and present during your final lab session as specified above.

Bonus:

Early completion Bonus: There is no early completion bonus for this lab

Other bonus tasks are detailed above.